

Case Study 5: Predicting Firewall Actions Using Support Vector Machines and Stochastic Gradient Descent

Kristin Henderson

July 6, 2025

1 Introduction

This study aims to predict network traffic actions using three classification models: a linear support vector machine (SVM), an SVM with a radial basis function (RBF) kernel, and a logistic regression model trained using stochastic gradient descent (SGD). A well-performing model that balances accuracy and speed could assist in filtering incoming network connection requests, supporting cybersecurity and firewall operations.

2 Data

2.1 Dataset

The Internet Firewall dataset used in this analysis comes from the University of California Irvine Machine Learning Repository. It contains 65,532 records of network traffic observed at a university firewall. Each record includes 11 features: 4 categorical features indicating port roles and 7 numerical features related to information flow. The multi-class target variable, **Action**, indicates the firewall’s decision: **allow**, **drop**, **deny**, or **reset-both**. The features are summarized in Table 1.

Table 1: Features Used in the Classification Models

Feature	Description
Source Port	Port number of the originating device
Destination Port	Port number of the destination device
NAT Source Port	Translated source port through Network Address Translation (NAT)
NAT Destination Port	Translated destination port through NAT
Bytes	Total bytes transmitted during the session
Bytes Sent	Number of bytes sent from source to destination
Bytes Received	Number of bytes received from destination
Packets	Total number of packets in the session
Elapsed Time (sec)	Duration of the session in seconds
pkts_sent	Number of packets sent
pkts_received	Number of packets received
Action	Target: Firewall decision. Possible values include: <i>allow</i> (connection permitted) <i>deny</i> (initial connection blocked) <i>drop</i> (packets silently discarded) <i>reset-both</i> (TCP session terminated from both source and destination)

The target classes are highly imbalanced: 57.44% of records are allowed, 22.87% are denied, 19.61% are dropped, and only 0.08% are reset-both, as shown in Figure 1.

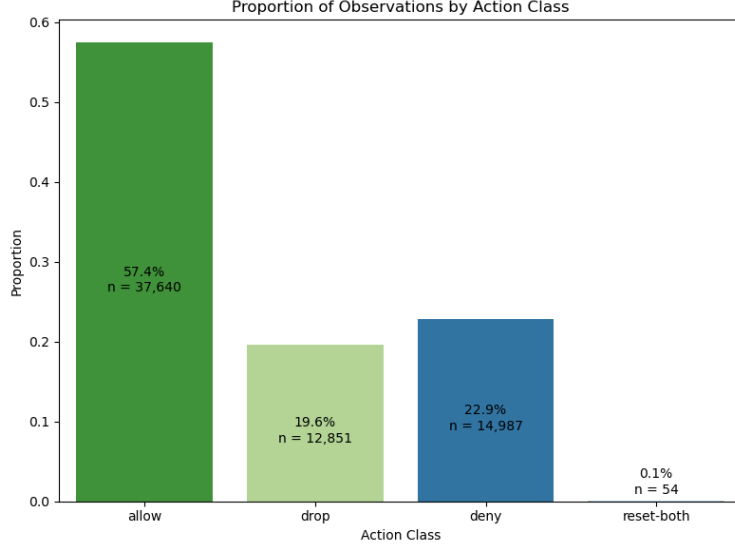


Figure 1: Proportion of observations in each action class. Counts and percentages are labeled on or above the bars.

2.2 Preprocessing

The dataset contains no missing values.

To better understand the port-related features and guide grouping decisions, the 20 most frequent values for each port column—`Source Port`, `NAT Source Port`, `Destination Port`, and `NAT Destination Port`—were plotted, stratified by action class. As shown in Figure 2, several class-specific patterns are noticeable.

- `allow` traffic is dominant on service ports like 80 (HTTP), 443 (HTTPS), and 53 (DNS), especially in `Destination Port` and `NAT Destination Port`.
- `deny` and `drop` actions are more common on remote-access or vulnerable services such as ports 22 (SSH), 23 (Telnet), 445 (SMB), and 5900 (VNC).
- Port 443 appears in `Source Port` more often with `deny` than `allow` actions—an unusual pattern, as this port is typically seen in the `Destination Port` field for HTTPS traffic.
- Port 0 frequently appears in `NAT Source Port` and `NAT Destination Port` among `deny`, `drop`, and `reset-both` classes, suggesting anonymized or missing NAT data.
- Ephemeral source ports (≥ 1024) with low frequency dominate `allow` traffic in `Source Port`.

Interestingly, all rows with `Action = drop` had `NAT Destination Port = 0`. Nearly all `deny` and `reset-both` rows showed the same. Most `allow` rows had matching NAT and non-NAT destination ports—except for 480 cases, 479 of which had `NAT Destination Port = 0`.

Each of the four port features had a large number of unique values, many of which were rare. To reduce the dimensionality introduced by one-hot encoding and to keep the total number of model features under approximately 1000 (especially important for the RBF SVM), low-frequency values were grouped into a single `infrequent` category. Thresholds were set individually: a stricter cutoff of 10 was used for `Source Port`, and a threshold of 5 for the other three features.

The decision to group aggressively on `Source Port` was also supported by domain knowledge: high-numbered ports (≥ 1024), which dominate this feature, typically correspond to ephemeral outbound client-side connections. These ports are common in `allow` traffic and tend to have high cardinality but low frequency—that is, wide variety with shallow support. While this grouping labeled nearly 87% of `Source Port` values as `infrequent`, it drastically reduced feature count while preserving patterns in the data.

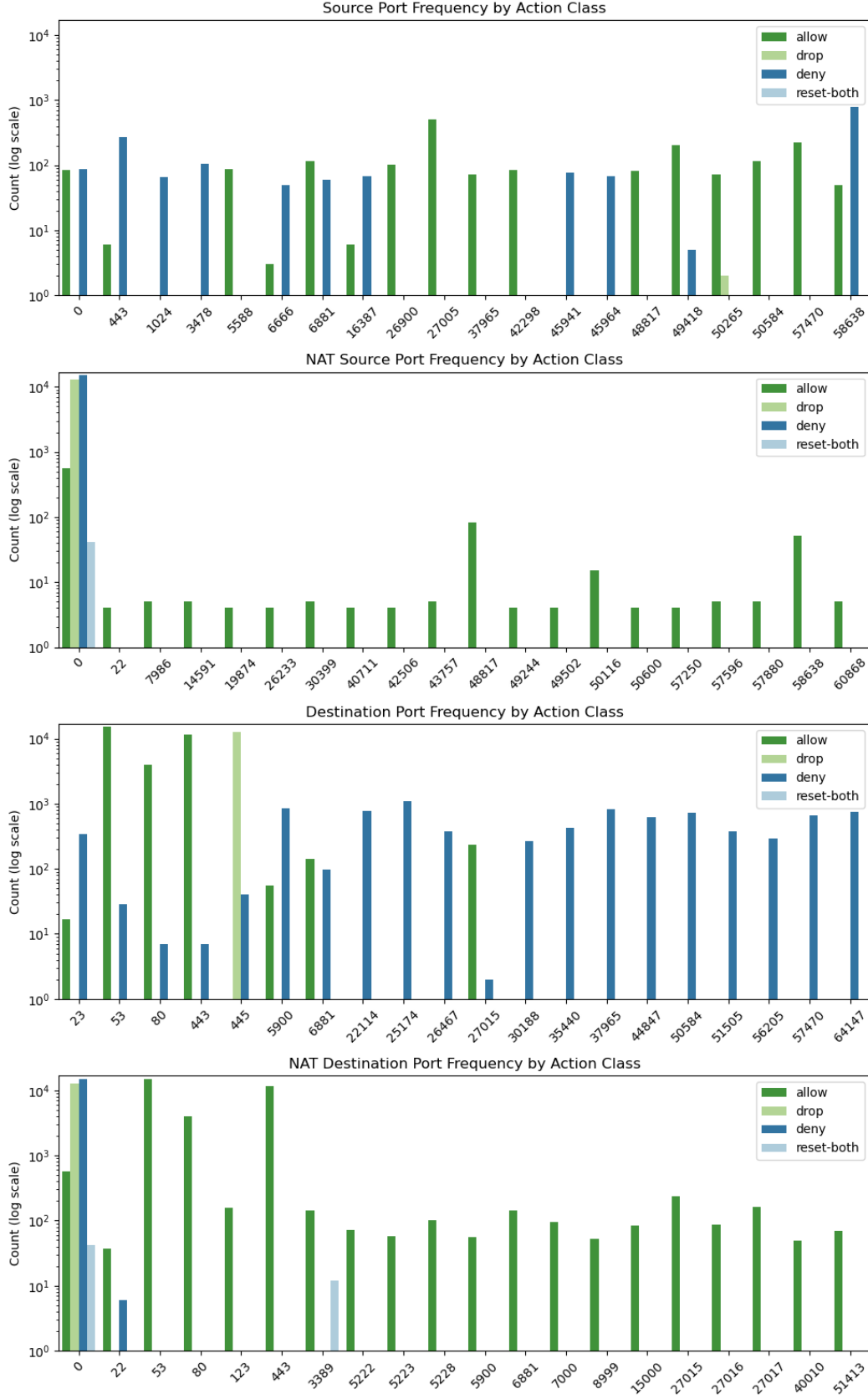


Figure 2: Barplot of the top 20 most frequent values for each port-related feature, stratified by Action class. Frequencies are displayed on a log scale to account for the wide range and skewed distribution of port usage counts.

Table 2 summarizes the observation thresholds, original and reduced category counts, and the percentage of rows labeled **infrequent** for each port feature.

Table 2: Port Feature Reduction via Infrequent Grouping

Port Feature	Threshold	Original Categories	After Grouping	% Infrequent Rows
Source Port	10	22,724	404	86.92%
NAT Source Port	5	29,152	12	56.33%
Destination Port	5	3,273	374	7.03%
NAT Destination Port	5	2,533	110	5.79%

After grouping infrequent values, the categorical values in each of the four port-related features were one-hot encoded, with the first category dropped as a reference level.

The numeric features were log-transformed because of extreme skewness in their distributions. Figure 3 shows the distributions of **Bytes**, **Packets**, and **Elapsed Time (sec)** by action class after log transformation. While log transformation reduced skewness, the distributions remained non-normal.

Log-Scaled Distributions of Representative Features by Action Class

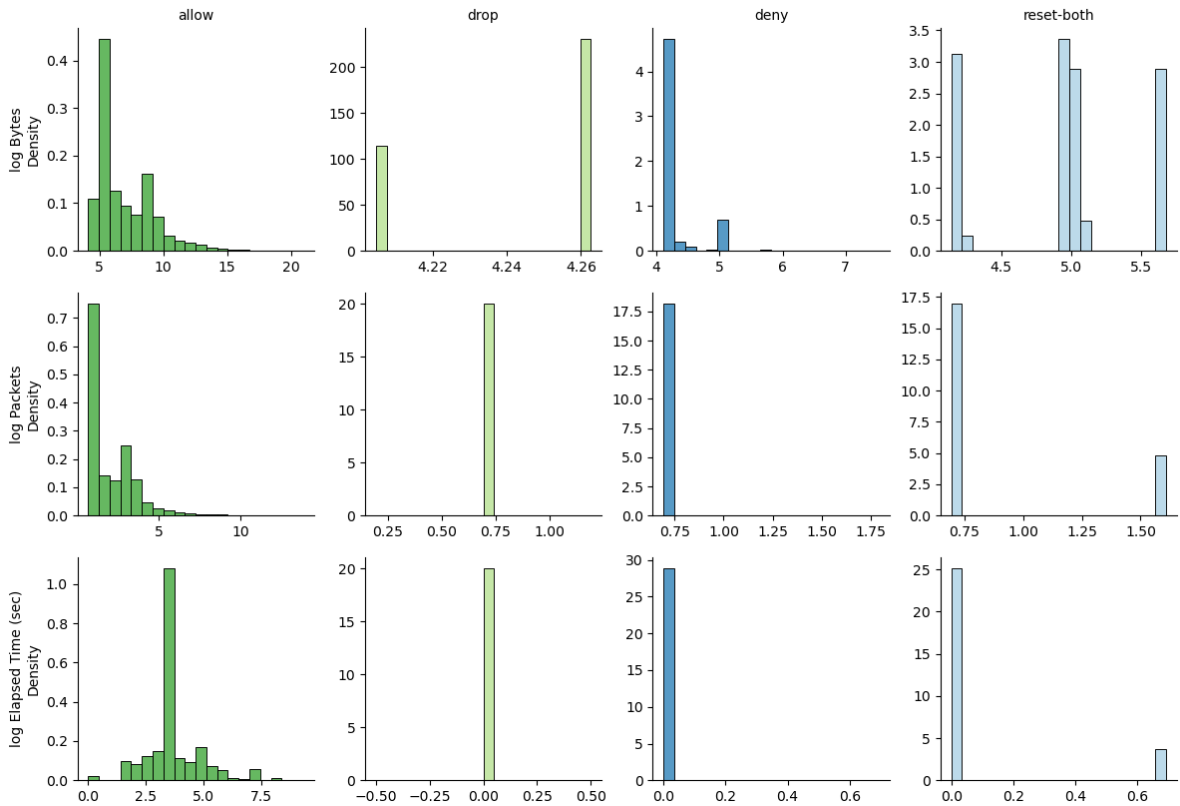


Figure 3: Histograms of representative features **Bytes**, **Packets**, and **Elapsed Time (sec)** for each action class after log transformation. Log transformation reduced the skewness of the distributions, but they remained non-normal.

The log-transformed features were then scaled using **StandardScaler**. Scaling is important for SVM and SGD-based logistic regression because both are sensitive to feature magnitudes. SVM maximizes the margin based on Euclidean distances, so features with larger ranges can distort the decision boundary. In

SGD, weight updates depend on the scale of the inputs, and scaling ensures that all features contribute proportionally during training.

3 Modeling

3.1 Linear SVM

Scikit-Learn's `LinearSVC()` was used to build a linear support vector machine (SVM) model with L2 regularization and a squared-hinge loss function. The `class_weight='balanced'` option was specified to account for class imbalance in the target.

The regularization parameter `C` was tuned over 20 logarithmically spaced values from `1e-6` to `1e6` using 5-fold stratified and shuffled cross-validation. A value of `C = 2.07` was selected because it achieved similarly high accuracy to nearby values, while providing stronger regularization and faster computation. Slightly higher accuracy (within one standard deviation) was observed at larger values of `C`, but this came at the cost of longer training time and weaker regularization.

To evaluate the model, `cross_val_predict()` was used with stratified and shuffled 5-fold cross-validation and the selected `C` value. This produced out-of-fold predictions used to generate a classification report and confusion matrix.

3.2 Nonlinear SVM (RBF Kernel)

A nonlinear SVM model was built using `SVC()` with an RBF kernel, which applies the kernel trick to map inputs into a higher-dimensional space. As before, the `class_weight='balanced'` option was used to address class imbalance in the target. A cache size of 400 (double the default of 200) was set to improve model training speed.

The regularization parameter `C` was tuned over the same parameter space, ranging from `1e-6` to `1e6`, but instead of cross-validation, a single 80/20 stratified and shuffled train/test split was used to reduce computational cost. A value of `C = 5.46e4` was selected because it achieved the highest accuracy and fastest computation among all tested values, while also providing stronger regularization than other models with comparable accuracy.

Model performance was evaluated on the 20% holdout test set using a classification report and a confusion matrix.

3.3 SGD Logistic Regression

`SGDClassifier()` with `loss='log_loss'` was used to build a logistic regression model with L2 regularization, trained using Stochastic Gradient Descent. As before, the `class_weight='balanced'` option was used to address class imbalance in the target.

The penalty term `alpha` was tuned in two stages: a coarse grid from `1e-6` to `1e0`, and a fine grid from `1e-5` to `1e-3`, both using 5-fold stratified and shuffled cross-validation. The best value, `alpha = 2.15e-4`, achieved the highest mean accuracy and lowest standard deviation with comparable training time.

The default maximum number of epochs (`max_iter=1000`) was used with `early_stopping=True` and a stopping criterion of 10 epochs without improvement (`n_iter_no_change=10`). The loss was evaluated on 20% of the shuffled CV training data (`validation_fraction=0.2`, `shuffle=True`). Using the selected `alpha`, along with `learning_rate='optimal'` and `tol=0.001`, the model converged in 11 epochs. Although the loss decreased slightly beyond epoch 1, it did not decrease beyond the tolerance threshold, triggering early stopping.

To encourage convergence in the 20-100 epoch range, `tol` values from `1e-3` to `1e-5` were tested. Accuracy remained stable, and average epochs increased only slightly, so the default tolerance was retained.

Learning rate strategies were also tested: `learning_rate='optimal'` versus `learning_rate='adaptive'` with `eta0` ranging from `1e-5` to `1e-1`. The best adaptive model used `eta0 = 1e-2` and trained for 106 epochs, aligning with the target range.

To select the final SGD model, both learning rate strategies were evaluated using `alpha = 2.15e-4` and `tol = 0.001`, with the `adaptive` model using the tuned `eta0 = 1e-2`.

Comparison of learning rate strategies for the SGD logistic regression model:

- Optimal LR: Log loss = 0.0341 ± 0.0055 , accuracy = 0.9966 ± 0.0004 , 11 epochs, avg. time = 0.151s
- Adaptive LR: Log loss = 0.0415 ± 0.0006 , accuracy = 0.9968 ± 0.0002 , 71 epochs, avg. time = 0.613s

The adaptive LR was slower and had slightly higher loss. For simplicity and efficiency, the default `optimal` setting was selected.

3.4 Parameter Summary

The key hyperparameters for each final model are summarized in Table 3.

Table 3: Final Hyperparameters for Each Selected Model

Model	Key Hyperparameters
Linear SVM (<code>LinearSVC()</code>)	<code>C = 2.07,</code> <code>loss='squared_hinge',</code> <code>penalty='l2'</code>
RBF SVM (<code>SVC()</code>)	<code>C = 5.46e4,</code> <code>kernel='rbf'</code>
SGD Logistic Regression (<code>SGDClassifier()</code>)	<code>alpha = 2.15e-4,</code> <code>loss='log_loss',</code> <code>learning_rate='optimal',</code> <code>early_stopping=True,</code> <code>tol=0.001,</code> <code>n_iter_no_change=10</code>

Note: All models were trained with `class_weight='balanced'` to address class imbalance.

4 Results

4.1 Confusion Matrices

To generate the confusion matrices shown in Figure 4, a default classification threshold of 0.5 was used for all models. Each matrix is shown with raw counts on the left and normalized values on the right to facilitate comparison between cross-validated and holdout models.

All three models achieved near-perfect accuracy on the `allow` class. For the two SVM models, fewer than 0.1% of `allow` observations were misclassified as `deny`, while the SGD model misclassified 0.27% as `deny` and fewer than 0.1% as `reset-both`.

The `deny` class was also classified with high accuracy—between 98% and nearly 100%—across all models. Fewer than 0.1% of `deny` observations were mislabeled as `allow`, with the remaining misclassifications split between `drop` and `reset-both`.

All three models perfectly predicted the `drop` class, with no misclassifications.

The minority class `reset-both`, which made up only 0.1% of the dataset (54 examples), was the most frequently misclassified. All misclassifications were as `deny`. The linear SVM and RBF SVM misclassified 22% and 27%, respectively, while the SGD model misclassified 48%.

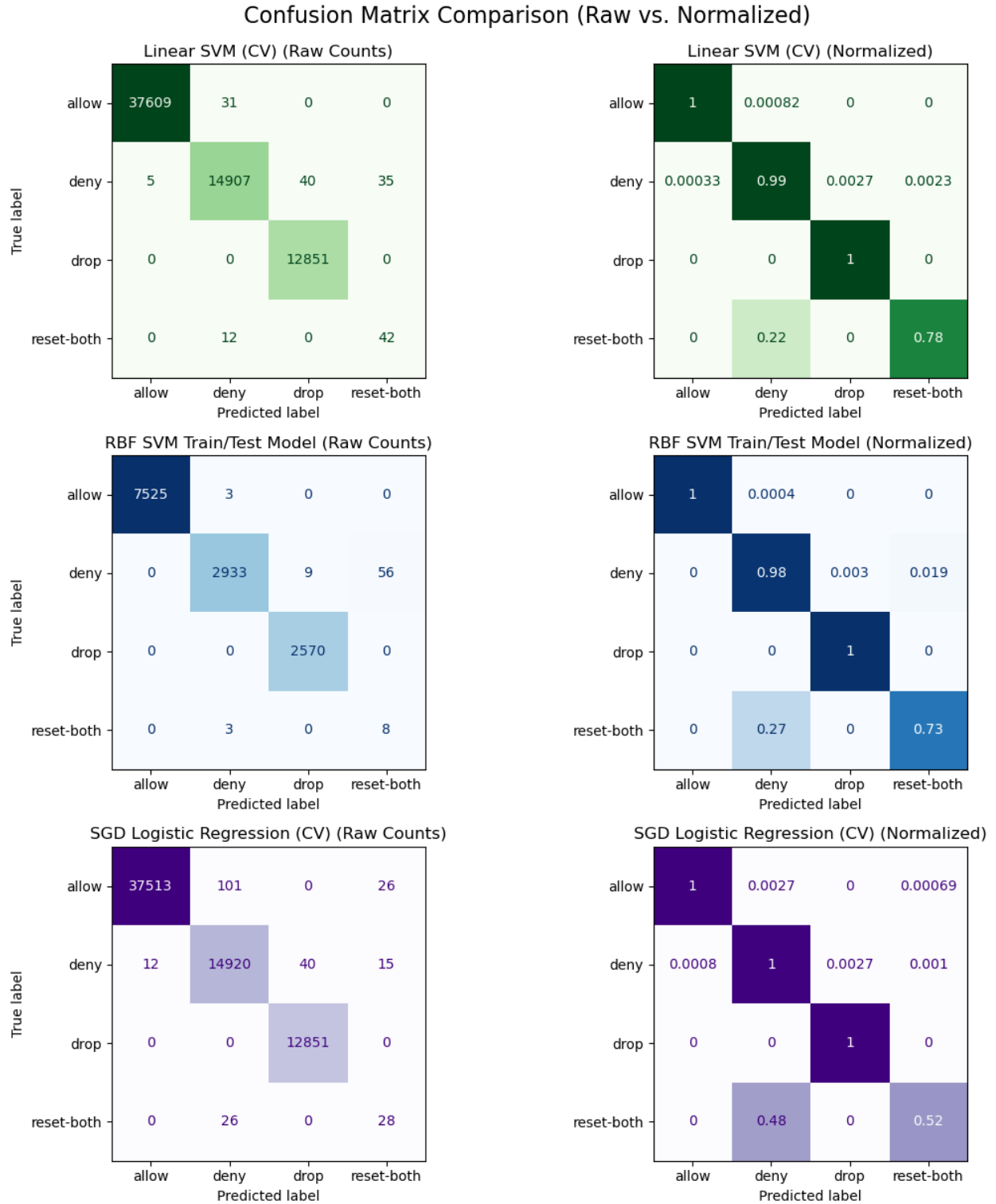


Figure 4: Confusion matrices showing classification results for the linear SVM (top row, green), RBF SVM (middle row, blue), and SGD logistic regression (bottom row, purple) models. Raw counts are shown on the left and normalized values on the right.

4.2 Overall Metrics

All three models achieved similarly high accuracy, as shown in Table 4, with linear SVM (0.9981 ± 0.0002) and SGD logistic regression (0.9966 ± 0.0004) exhibiting slightly higher and more stable performance across cross-validation folds compared to the RBF SVM (0.9946), which was evaluated on a single holdout test set.

Macro-averaged metrics evaluate performance across all classes equally, giving the same weight to minority classes regardless of their frequency. In contrast, weighted metrics adjust for class frequency when averaging across classes. Whether macro or weighted metrics are more appropriate depends on the analytic goal, whether to prioritize minority classes or overall dataset performance.

The linear SVM achieved the highest macro and weighted scores overall, reflecting the strongest and most consistent performance across all classes. As expected, the SGD logistic regression model ran faster than either SVM model, with the RBF SVM being the slowest. The SGD model outperformed the RBF SVM in all metrics except for macro recall and weighted precision. The macro metrics tend to be lower than the weighted metrics due to lower performance on the minority class, **reset-both**.

Table 4: Overall Classification Metrics and Modeling Time by Model

Metric	Linear SVM (CV)	RBF SVM (Holdout)	SGD Logistic Regression (CV)
Accuracy	0.9981	0.9946	0.9966
Macro Precision	0.8848	0.7799	0.8485
Macro Recall	0.9429	0.9263	0.8777
Macro F1-score	0.9088	0.7999	0.8614
Weighted Precision	0.9983	0.9981	0.9968
Weighted Recall	0.9981	0.9946	0.9966
Weighted F1-score	0.9982	0.9961	0.9967
Modeling Time (s)	0.37	0.84	0.18

Note: Modeling times include both training and prediction. Times for cross-validated models reflect the mean per fold. RBF SVM time is based on a single train/test split.

4.3 Per-Class Metrics

As shown in Table 5, all models achieved at least 99.5% precision, recall, and F1-score on the **allow** and **drop** classes. The linear SVM also reached at least 99.5% on all three metrics for the **deny** class.

For **deny**, the RBF SVM showed the highest precision (99.8%) but lower recall (97.8%) and F1-score (98.8%) than the other models. The SGD model had slightly lower precision (99.2%) but higher recall (99.6%).

The minority class **reset-both** was the most challenging for all models. The linear SVM achieved the best performance: precision (54.6%), recall (77.8%), and F1-score (64.1%). The SGD model lagged behind the others models with lower recall of 51.9%, while the RBF SVM showed very low precision (12.5%) despite a relatively high recall (72.7%).

Table 5: Per-Class Classification Metrics for Each Model

Model	Class	Precision	Recall	F1-score	Support
Linear SVM (CV)	allow	0.9999	0.9992	0.9995	37,640
	deny	0.9971	0.9947	0.9959	14,987
	drop	0.9969	1.0000	0.9984	12,851
	reset-both	0.5455	0.7778	0.6412	54
RBF SVM (Holdout)	allow	1.0000	0.9996	0.9998	7,528
	deny	0.9980	0.9783	0.9880	2,998
	drop	0.9965	1.0000	0.9983	2,570
	reset-both	0.1250	0.7273	0.2133	11
SGD Logistic Regression (CV)	allow	0.9997	0.9966	0.9982	37,640
	deny	0.9916	0.9955	0.9935	14,987
	drop	0.9969	1.0000	0.9984	12,851
	reset-both	0.4058	0.5185	0.4553	54

5 Conclusion

All three models achieved high accuracy, as well as strong precision, recall, and F1-scores across most classes—except for the minority class, **reset-both**, which was challenging for all. Because both linear models produced slightly better overall metrics and ran more quickly, the results suggest that a linear model is sufficient for this application.

The linear SVM was slightly slower than the SGD model, but it achieved the highest overall accuracy and the best per-class metrics, including on the **reset-both** class. It was also the easiest model to tune. Given the size of this dataset, either linear model would be a strong choice for optimizing performance. However, with larger datasets, the SGD model would likely offer a better balance of speed, scalability, and performance.